PROJECT ACTIVITY REPORT

Activity 4.3: Engagement Scoring Logic

Project: Multimodal Attendance & Engagement Agent

1. Activity Overview & Objective

This activity turns fused multimodal signals into **interpretable engagement scores**. Rule-based logic and predefined thresholds assess engagement across different time segments of the session. The factors considered together are sustained visual presence, movement frequency, inactivity duration and speech activity intensity.

Temporal engagement profiling. Scores are computed at regular intervals to build a temporal profile instead of one aggregated value. This makes attention fluctuations, peak engagement periods and extended disengagement phases visible. The logic is transparent and adjustable, so it can be tuned to classroom requirements.

2. Multimodal Scoring Factors & Threshold Matrix

To avoid black-box ambiguity, the scoring engine evaluates four behavioral dimensions taken from the synchronized multimodal stream at fixed time segments. Each factor is checked against configurable thresholds before the interval composite score is computed.

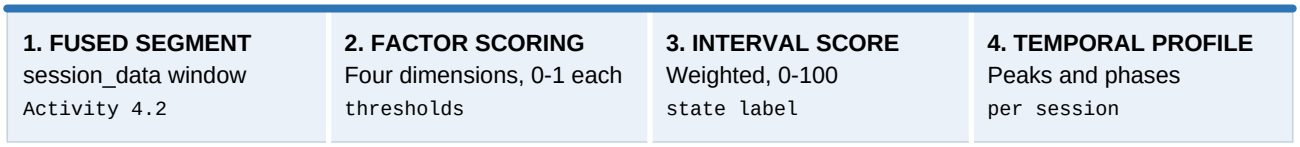


Figure 1: Engagement scoring pipeline.

Scoring factor	Source modality	Predefined threshold logic	Impact on interval score
Sustained visual presence	Vision Module	Continuous face / body detection ratio >= 0.80 within the active time window.	Sets baseline attendance stability and a positive attention weight.
Movement frequency	Vision Module	Moderate pose transitions (0.15 - 0.55) vs. excessive restlessness (> 0.65).	Rewards natural attentive posture shifts; penalizes erratic fidgeting or distraction.
Inactivity duration	Vision + Audio	Extended static posture with zero acoustic interaction (> 45 s).	Applies a progressive decay penalty to flag passive zoning-out or prolonged disengagement.
Speech activity intensity	Audio Module	Structured vocal energy and VAD ratio aligned with classroom dialogue thresholds.	Boosts interval scores during active Q&A, lectures or collaborative discussion.

2.1 Score Formula

Each factor is first converted to a 0-1 value. The interval score is the weighted sum of the three positive factors minus the inactivity penalty, scaled to 0-100:

```
score = 100 * ( w_presence * presence
              + w_movement * movement_score(movement_frequency)
              + w_speech   * speech_intensity
              - w_inactive * inactivity_penalty ) / (w_presence + w_movement + w_speech)

Default weights:  presence 0.35 | movement 0.20 | speech 0.25 | inactivity penalty 0.20
State labels:     score >= 75  Peak Engagement
                  score <= 40  Disengaged
                  otherwise    Moderate Engagement
```

3. Reference Implementation (backend/core/engagement_scorer.py)

`EngagementScorer` computes interval-level scores and builds a full temporal session profile that exposes peak engagement windows, attention fluctuations and extended disengagement phases for downstream teaching suggestions, student analytics and session summaries.

3.1 Review of the Source Listing

The listing in the source document is close to this design, but checking it against the threshold matrix and running it on test inputs showed points that change the results. The completed implementation in 3.2 addresses each of them.

Observation in source listing	Effect	Change in 3.2
The three positive weights add up to 0.80 (0.35 + 0.20 + 0.25); the 0.20 inactivity weight is only subtracted.	A perfect interval scores 80, not 100 (checked with test input), so the Peak threshold of 75 is almost unreachable.	Score is divided by the sum of positive weights, so a perfect interval scores 100. Set normalize=False to reproduce the original scale.
Movement uses 1 - abs(movement - 0.35).	Zero movement still earns 0.65, and full restlessness (1.0) still earns 0.35, which does not match the 0.15 - 0.55 band and > 0.65 penalty in the matrix.	Band-based movement score that follows the matrix (see table below).
Inactivity is a plain ratio.	The matrix's 45 s rule with progressive decay is not implemented.	Optional inactivity_seconds input: no penalty up to 45 s, then rising to full penalty over the next 45 s. The ratio input still works.
Thresholds are hard-coded; only weights can be configured.	The 'transparent and adjustable' goal is only partly met.	Thresholds, bands and phase length all come from the config dictionary.
The listing ends mid-function and a key reads 'disengagement phases' (with a space).	The profile is incomplete and the key is awkward to use.	Completed; key is disengagement_phases and consecutive Disengaged intervals are grouped into phases.
An empty segment list returns a mean score of 0.0.	An empty session looks like total disengagement.	Returns None for the mean and volatility.

Movement factor: source formula vs. threshold-matrix version

Movement frequency	0	0.15	0.35	0.55	0.65	0.80	1.00
Source: 1 - |m - 0.35|	0.65	0.80	1.00	0.80	0.70	0.55	0.35
Matrix-based (3.2)	0.00	1.00	1.00	1.00	0.70	0.40	0.00

3.2 Completed Implementation

File: backend/core/engagement_scorer.py (reference implementation)

```
# File: backend/core/engagement_scorer.py
DEFAULT_CONFIG = {
    "weights": {"presence": 0.35, "movement": 0.20,
                "inactivity_penalty": 0.20, "speech_intensity": 0.25},
    "peak_threshold": 75.0,
    "disengaged_threshold": 40.0,
    "movement_band": (0.15, 0.55),   # natural posture shifts
    "restless_above": 0.65,          # excessive fidgeting
    "inactivity_grace_sec": 45.0,    # static + silent time tolerated
    "inactivity_span_sec": 45.0,     # extra seconds until full penalty
    "min_phase_len": 2,              # intervals that make a disengagement phase
    "normalize": True,               # scale so a perfect interval scores 100
}


def _clip(x, lo=0.0, hi=1.0):
    return max(lo, min(hi, x))


class EngagementScorer:
    def __init__(self, config: dict = None):
        cfg = {**DEFAULT_CONFIG, **(config or {})}
        cfg["weights"] = {**DEFAULT_CONFIG["weights"], **cfg.get("weights", {})}
        self.cfg = cfg
        self.weights = cfg["weights"]
        self.peak_threshold = cfg["peak_threshold"]
        self.disengaged_threshold = cfg["disengaged_threshold"]

    # --- factor scoring (each returns 0.0 - 1.0) --------------------------
    def _movement_score(self, m):
        lo, hi = self.cfg["movement_band"]
        restless = self.cfg["restless_above"]
        if m < lo:                       # too static: partial credit
            return m / lo
        if m <= hi:                      # natural attentive posture shifts
            return 1.0
        if m <= restless:                # drifting toward restlessness
            return 1.0 - 0.3 * (m - hi) / (restless - hi)
        return 0.7 * (1.0 - (m - restless) / (1.0 - restless))   # fidgeting

    def _inactivity_penalty(self, seg):
        if "inactivity_seconds" in seg:  # static posture AND no speech
            over = seg["inactivity_seconds"] - self.cfg["inactivity_grace_sec"]
            return _clip(over / self.cfg["inactivity_span_sec"])   # progressive
```

```
        return _clip(seg.get("inactivity_duration_ratio", 0.0))

    # --- interval score ------------------------------------------------
    def compute_interval_score(self, segment: dict) -> dict:
        presence = _clip(segment.get("sustained_presence", 0.0))
        movement = self._movement_score(_clip(segment.get("movement_frequency", 0.0)))
        speech = _clip(segment.get("speech_activity_intensity", 0.0))
        penalty = self._inactivity_penalty(segment)

        w = self.weights
        parts = {"presence": presence * w["presence"],
                 "movement": movement * w["movement"],
                 "speech": speech * w["speech_intensity"],
                 "inactivity_penalty": -penalty * w["inactivity_penalty"]}
        scale = w["presence"] + w["movement"] + w["speech_intensity"]
        factor = 100.0 / scale if self.cfg["normalize"] else 100.0
        score = round(_clip(sum(parts.values()) * factor, 0.0, 100.0), 2)

        state = ("Peak Engagement" if score >= self.peak_threshold
                 else "Disengaged" if score <= self.disengaged_threshold
                 else "Moderate Engagement")
        return {"timestamp": segment.get("timestamp"), "score": score, "state": state,
                "components": {k: round(v * factor, 2) for k, v in parts.items()}}

    # --- temporal profile ----------------------------------------------
    def _phases(self, timeline):
        phases, run = [], []
        for item in timeline + [None]:               # sentinel closes the last run
            if item and item["state"] == "Disengaged":
                run.append(item)
                continue
            if len(run) >= self.cfg["min_phase_len"]:
                phases.append({"start": run[0]["timestamp"],
                               "end": run[-1]["timestamp"], "intervals": len(run)})
            run = []
        return phases

    def generate_temporal_profile(self, segments: list) -> dict:
        ordered = sorted(segments, key=lambda s: s.get("timestamp", 0))
        timeline = [self.compute_interval_score(s) for s in ordered]
        scores = [t["score"] for t in timeline]
        if not scores:
            return {"temporal_profile": [], "mean_score": None, "peak_periods": [],
                    "disengagement_phases": [], "volatility": None}
```

```
        changes = [abs(b - a) for a, b in zip(scores, scores[1:])]
        return {
            "temporal_profile": timeline,
            "mean_score": round(sum(scores) / len(scores), 2),
            "min_score": min(scores), "max_score": max(scores),
            "volatility": round(sum(changes) / len(changes), 2) if changes else 0.0,
            "peak_periods": [t["timestamp"] for t in timeline
                             if t["state"] == "Peak Engagement"],
            "disengagement_phases": self._phases(timeline),
        }


def segment_from_session_data(sd: dict) -> dict:
    """Adapter: Activity 4.2 session_data -> scorer segment."""
    v, a = sd.get("vision") or {}, sd.get("audio") or {}
    speech = a.get("speech_ratio", 0.0)
    return {
        "timestamp": sd["window"]["start"],
        "sustained_presence": v.get("presence_ratio", 0.0),
        "movement_frequency": v.get("mean_motion", 0.0),
        # static AND silent share of the window
        "inactivity_duration_ratio": v.get("inactivity_ratio", 0.0) * (1 - speech),
        "speech_activity_intensity": speech,
    }
```

3.3 Adapter from Activity 4.2

`segment_from_session_data()` (end of the listing above) converts the fused window from Activity 4.2 into a scorer segment. The mapping is approximate: `mean_motion` stands in for movement frequency, and inactivity is the static share of the window multiplied by the silent share, so only time that is both static and without speech counts as inactive.

3.4 Behaviour Check

The scorer was run on eight synthetic 30-second intervals (a class that starts attentive, drifts into a silent and static phase, then recovers). This is test data, not a real session.

Time (s)	Score	State	Note
0	87.5	Peak Engagement	Full presence, natural movement, active speech.
30	84.38	Peak Engagement	
60	70.0	Moderate Engagement	Speech and presence start to drop.
90	16.04	Disengaged	Low presence, static, silent.
120	0.83	Disengaged	
150	0.83	Disengaged	Three consecutive intervals form one disengagement phase.
180	86.25	Peak Engagement	Recovery.
210	94.69	Peak Engagement	

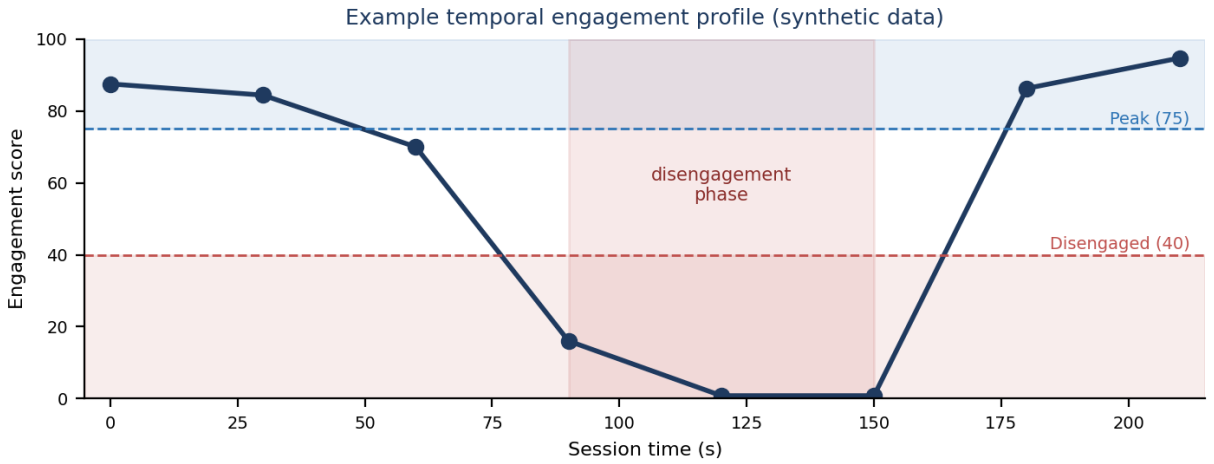


Figure 2: Profile produced from the test data above. Mean 55.06, volatility 25.79, one disengagement phase (90 s - 150 s), peak periods at 0, 30, 180 and 210 s.

Each result also carries a `components` breakdown (points from presence, movement, speech and the inactivity penalty) so an instructor can see why an interval scored as it did. For the 90 s interval above: presence +21.88, movement +16.67, speech 0.00, inactivity penalty -22.50.

4. Downstream Integration

- **Teaching suggestions:** real-time detection of attention fluctuations or an extended disengagement phase triggers immediate pedagogical prompts for the instructor. For example, a new entry in `disengagement_phases` can be passed to the suggestion engine from Activity 4.2.
- **Student analytics and session summaries:** interval scores and peak periods are stored so post-session trend charts and participation reports can be generated.
- **Storage shape:** the profile is plain JSON-serializable data (a timeline list plus summary fields), so it can be written to a file or database as it is.

Wiring fusion, scoring and suggestions

```
from backend.core.signal_fusion import SignalFusion
from backend.core.engagement_scorer import EngagementScorer, segment_from_session_data

windows = fusion.fuse_stream(vision, audio, start, end, expected_count)   # Activity 4.2
segments = [segment_from_session_data(w) for w in windows]
profile = EngagementScorer().generate_temporal_profile(segments)

for phase in profile["disengagement_phases"]:
    suggestions.generate_suggestions({"context": "disengagement_risk", **phase})
```

5. Configuration Reference

Setting	Default	Effect
weights	presence 0.35, movement 0.20, speech 0.25, inactivity 0.20	Relative importance of each factor. Weights need not sum to 1 when normalize is on.
peak_threshold / disengaged_threshold	75 / 40	Score cut-offs for Peak and Disengaged states.
movement_band / restless_above	0.15 - 0.55 / 0.65	Range of natural posture shifts and the start of fidgeting.
inactivity_grace_sec / inactivity_span_sec	45 s / 45 s	Static, silent time tolerated, then the time over which the penalty rises to full.
min_phase_len	2 intervals	Consecutive Disengaged intervals needed to count as a phase.
normalize	True	Scales scores so a perfect interval is 100.

Tuning guidance

- Choose the interval length (for example 30 s) so that the 45 s inactivity rule spans more than one interval; if intervals are longer than 45 s, a single interval can reach the full penalty.
- Start from the defaults, review the profile of a few recorded lessons with the teacher, then adjust thresholds. Change one setting at a time and keep a record of the values used.
- For a lecture-style class, consider lowering the speech weight, since students listening silently are not disengaged.

6. Limitations & Responsible Use

Limitation	Impact	Mitigation
Speech is a positive factor	Quiet, focused work scores lower than a discussion.	Lower the speech weight for quiet lesson types, or use the context label from Activity 4.2 to adjust interpretation.
Thresholds and weights are starting values	Scores may not match real classroom experience.	Calibrate against teacher-labelled recordings.
Class-level, camera-based signals	Not a measure of any individual's learning or effort.	Use for trends and prompts; do not use for grading or discipline.
Movement and posture are indirect proxies	Neurodiverse or tired students may be misread.	Keep a human in the loop and show the score components.
Face and audio data are personal data	Privacy and consent obligations.	Obtain consent; store derived scores rather than raw media where possible.

7. Validation Checklist

Tick each item after running it on your machine:

Check	How to verify	Status
Perfect interval scores 100	Presence 1, natural movement, full speech, no inactivity	[ ]
Restlessness is penalized	Movement 1.0 gives a clearly lower score than 0.35	[ ]
45 s rule works	Inactivity of 45 s costs nothing, 90 s gives the full penalty	[ ]
Disengagement phase detected	Three Disengaged intervals in a row appear as one phase	[ ]
Empty session handled	No segments: mean is None, no error	[ ]
Config changes take effect	Change a threshold and re-run: states change as expected	[ ]
Teacher review	Profile of a real lesson matches the teacher's impression	[ ]